

WRRRA Game 1.0

Minimal Sufficient State for Complex Games and a Playable Go Engine

Constructive Omok and Go Analysis, Exhaustive Validation, and WRRRA Go 0.1

Wonsik Choi

Independent Researcher

janefather@gmail.com

17 September 2026

WRRRA Core 1.0 remains frozen

Integrated technical report and reproducibility archive

Abstract

WRRR Game 1.0 connects three levels of evidence under one unchanged execution contract. WRRR Game 0.1 computes a constructive forced-win phenotype in 9×9 freestyle Omok and a simple-ko counterexample in 5×5 Go. WRRR Game 0.2 exhaustively enumerates 6,036,001 reachable states of a 4×4 Connect-K system and 132,161 reachable complete states of 3×3 Go. The certified Renderer reduction has zero minimax-value mismatches over the entire Connect-K reachability graph, while 752 visible-state equivalence classes in Go have legal-action sets that depend on previous-board residue. WRRR Go 0.1 then reuses the verified Go transition system in a playable 9×9 engine, combining an explicit eleven-channel Renderer with Monte Carlo Tree Search and an auditable decision ledger.

The central methodological claim is precise: a game agent should preserve the minimum state required for exact legality and state update, then spend bounded computation on structurally informative legal continuations. Residue does not predict or preserve the future. It carries present constraint information that is absent from the visible board. Search generates and compares successor states permitted by the current rules. The archive includes source code, deterministic rule tests, exhaustive-search ledgers, paired-colour control matches, and a complete SGF self-play record.

Keywords: WRRR; Go; Baduk; Omok; Connect K; MCTS; minimal sufficient state; residue; Renderer; exhaustive search; explainable game AI; complex systems.

Executive result

Stage	Question	Executed evidence	Established meaning
Game 0.1	Can the structure compute concrete cases?	70 Omok replies; one ko counterexample	Constructive operation
Game 0.2	Are value and legality preserved globally?	6,036,001 Connect-K states; 132,161 Go states	Exhaustive finite validation
Game 1.0	Can the same structure select actual moves?	Playable 9×9 engine and paired controls	Transition from analysis to action

Domain	Scope	Result	Interpretation
Connect K	6,036,001 reachable states	0 minimax mismatches	Certified reduction preserves value
Connect K	Empty-board alpha-beta	245,560 → 32 nodes	Same exact root value
3×3 Go	132,161 complete states	752 residue-dependent classes	Board alone is insufficient
WRRR Go	16 games vs uniform MCTS	15–1	Equal simulation budget
WRRR Go	16 games vs random	16–0	Alternating colours
9×9 self-play	82 moves	White wins; Black margin -8.5	two consecutive passes

These match results are an action-selection evaluation, not a claim of professional Go strength. Their importance is that the same explicit legality, residue, transition, Renderer, and ledger structure runs end to end and can be compared with controls under equal computational budgets.

Contents

1. The common WRRR execution contract and exact terminology
2. WRRR Game 0.1: constructive Omok and Go analyses
3. WRRR Game 0.2: exhaustive validation
4. WRRR Go 0.1: a playable action-selection algorithm
5. Execution tests, paired controls, and 9×9 self-play
6. Scientific value, limits, and generalization
7. Reproducibility, file manifest, and release metadata

1. Common execution contract

1.1 A domain profile, not a modification of WRRRA Core 1.0

The game work is a domain profile, not a revision of WRRRA Core 1.0. The frozen Core fixes the execution order—SOURCE, LAW, STATE and RESIDUE, BOUNDARY, COMMON CARRIER, UPDATE, RENDERER, PHENOTYPE, and LEDGER—while each game's rules and objectives remain in its profile. Omok and Go keep their real differences but face the same diagnostic questions.

Core item	Omok / Connect K	Go	Playable Go engine
SOURCE	Placement on an empty point	Placement or pass	Human or algorithmic action
LAW	Alternation and K-in-a-row	Liberty, capture, suicide, simple ko	Executable rule functions
STATE	Board and side to move	Board, turn, residue, pass count	Adds move index and tree nodes
RESIDUE	No additional history	One previous board	Used directly for ko legality
BOUNDARY	Empty points and winning lines	Boundary of legal placements	Only legal candidates are searched
CARRIER	Finite grid	Finite orthogonal grid	Shared coordinates and occupancy
UPDATE	Place and alternate	Capture, ko test, then alternate	Immutable GoState result
RENDERER	Win threats and forced blocks	Capture, liberty, connection, cut	Prior and rollout ordering
PHENOTYPE	Win, forced loss, open search	Capture, rescue, atari, and others	Human-readable move label
LEDGER	State values and nodes	Legal sets and witness histories	Visits, values, features, record

1.2 State sufficiency

A state representation M is sufficient only if two legal histories compressed to the same M admit the same legal actions and compatible next-state updates. If identical visible boards permit different

actions because of history, M omits information required by the current computation. This is the criterion for adding residue; it is not a reason to store the entire past indiscriminately.

$$M(h_1) = M(h_2) \Rightarrow A_{legal}(h_1) = A_{legal}(h_2)$$

Freestyle Omok needs the board and side to move. Go with simple ko additionally needs one previous board to reject immediate repetition. If the selected law uses positional superko, residue must grow to encode the longer repetition constraint. The law, not preference, determines the minimum memory.

1.3 Interpretation, search, and prediction

Term	Meaning in this work	Meaning explicitly excluded
State transition	Compute a next state after a legal action	Observe reality's future in advance
Successor branch	A candidate generated under current rules	An event that must actually occur
Search	Compare candidate states under an objective	Automatic empirical prediction
Residue	Current constraint information needed for legality	Storage of the future itself
Renderer score	Structural features and search priority	An unconditional prophecy of the game result

Accordingly, this report replaces earlier phrases such as ‘preserving future branches.’ The exact statement is: residue preserves the information needed to compute legal actions and exact state updates; Renderer-guided search generates and evaluates rule-permitted successors.

2. WRRRA Game 0.1: constructive analysis

2.1 A forced-win outlet in 9×9 freestyle Omok

Game 0.1 evaluates every one of 71 legal candidates in a specified 9×9 position. Only the central move (4, 4) creates four immediate winning outlets for Black's following turn. All 70 legal White replies were then applied; none removed all four outlets. This is a constructive case in which a Renderer exposes a living boundary and a multiple-threat phenotype before deeper search.

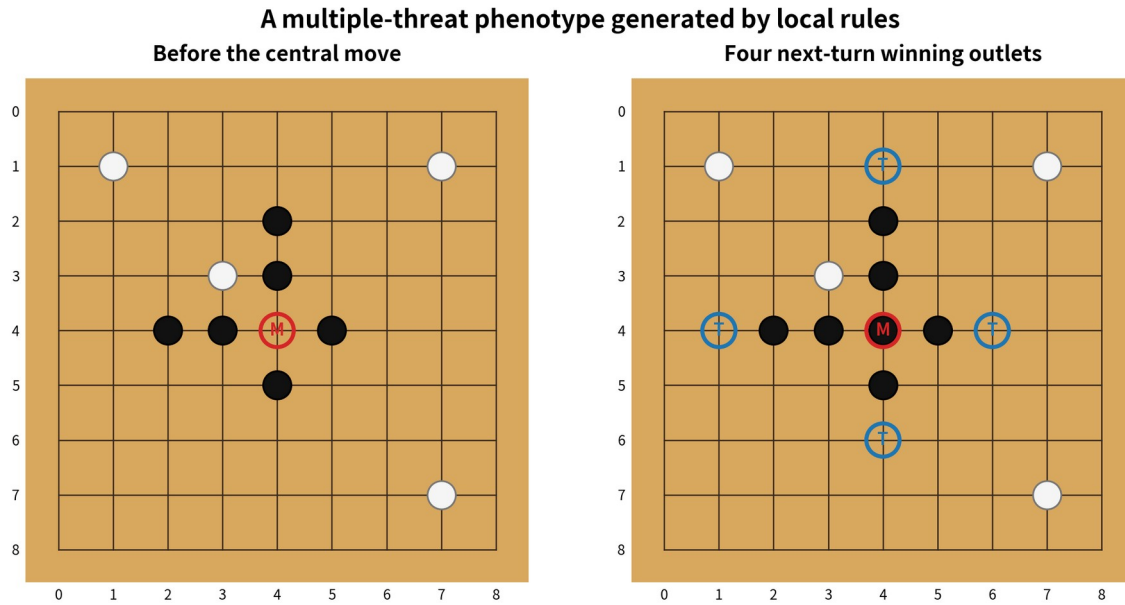


Figure 1. The central Omok move creates four next-turn winning outlets.

Item	Value	Meaning
Legal candidates	71	All empty points
Multiple-outlet candidates	1	Only the central move
Winning outlets created	4	Four endpoints on two lines
Defensive replies tested	70	Every legal White response
Replies removing all outlets	0	Forced-win certificate
Candidate reduction	98.59%	Constructive Renderer example

2.2 Simple ko and residue in 5×5 Go

The Go example constructs a direct counterexample to board-only state. A capture removes one White stone. Immediate recapture would reproduce the previous board and is illegal under simple ko. With previous-board residue the position has 18 legal actions; without it the forbidden recapture enters the set, producing 19.

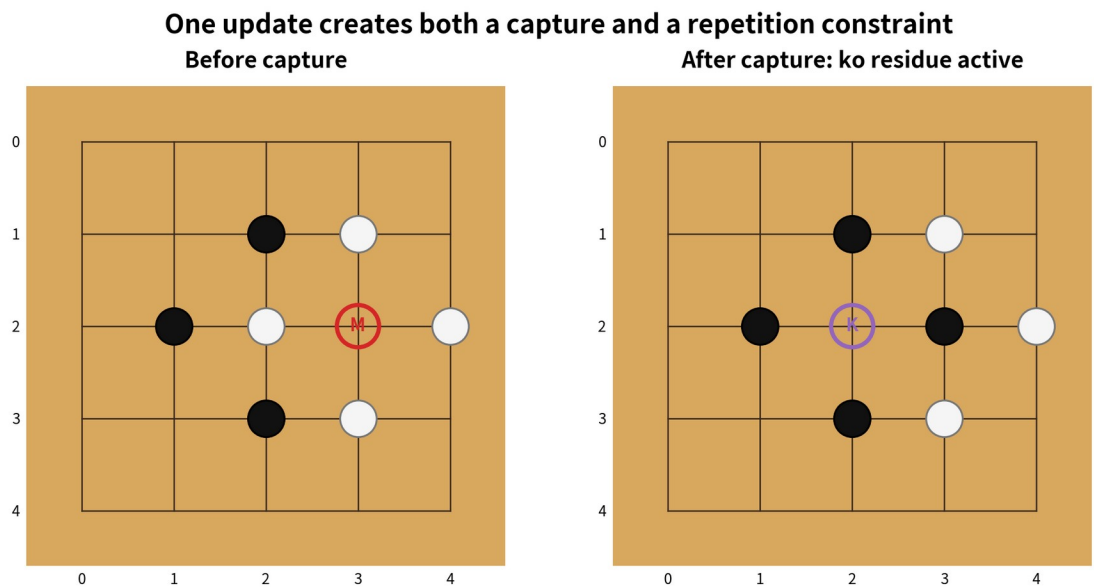


Figure 2. Previous-board residue blocks an immediate ko recapture.

Test	With residue	Without residue	Judgment
Immediate recapture	Illegal	Legal	The latter restores the prior board
Number of legal actions	18	19	One-action difference

Game 0.1 therefore ties two abstract terms to executable cases. Renderer exposes live threats and boundaries so that candidates can be ranked or safely reduced. Residue attaches a legal constraint that is absent from the current board. Game 0.2 asks whether these observations remain valid over complete reachability graphs.

3. WRRRA Game 0.2: exhaustive validation

3.1 The 4×4, three-in-a-row Connect-K system

Instead of claiming exhaustive coverage of standard 15×15 Omok, the experiment uses a fully enumerable system: a 4×4 board where three in a row wins. Starting from the empty board, it closes only states reachable under alternation and early termination. The question is not playing strength on a small board. It is whether a certified Renderer reduction changes the exact minimax value at any reachable state.

Ledger item	Value	Ledger item	Value
Reachable states	6,036,001	Directed legal edges	23,453,344
Terminal wins	2,572,442	Terminal draws	18
Immediate-win states	3,143,087	Single mandatory blocks	65,828
Multiple-threat forced losses	232,299	Open-search states	22,327
All-state value mismatches	0	Empty-board value	1

$$\sum_{S \in S_{reach}} \mathbf{1}[V_{base}(S) \neq V_{WRRRA}(S)] = 0$$

The reduction keeps only an immediate winning move when one exists; keeps the unique block when the opponent has exactly one immediate winning point; and issues a forced-loss certificate when two distinct winning points cannot both be blocked by one move. In every other state it discards nothing and changes only move order. Comparison with baseline minimax over all 6,036,001 reachable states yields zero value mismatches.

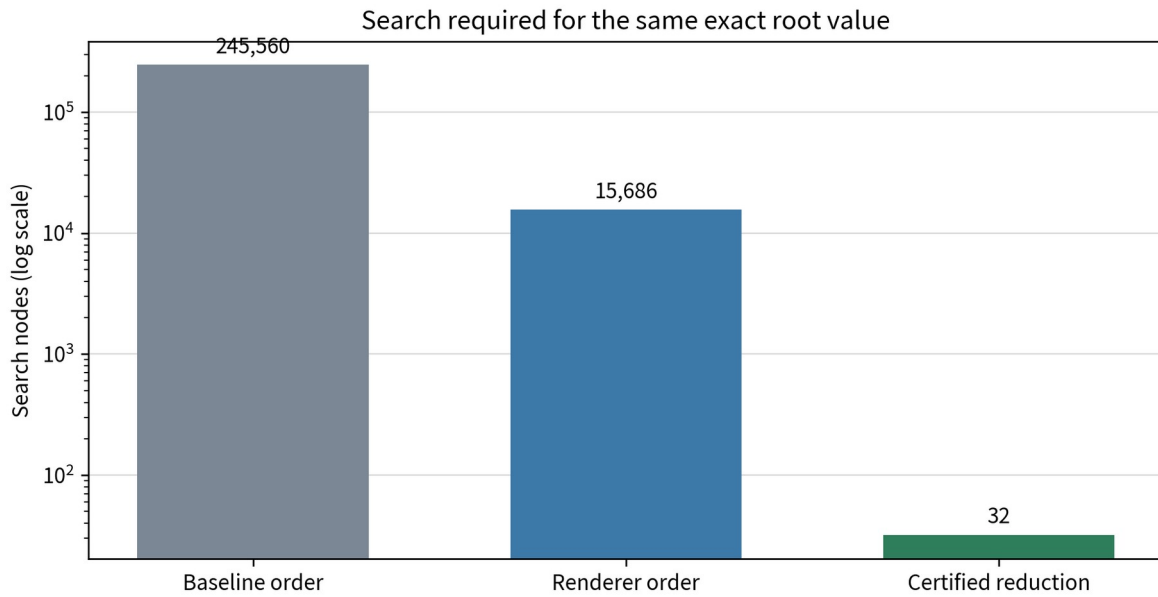


Figure 3. Alpha-beta nodes needed to recover the same exact empty-board value.

Search	Visited nodes	Reduction from baseline	Root value
Baseline row-major	245,560	0%	1
Renderer ordering	15,686	93.612%	1
Certified Renderer reduction	32	99.98697%	1

3.2 The complete reachable graph of 3×3 simple-ko Go

The Go system implements capture, suicide prohibition, simple ko, pass, and termination after two consecutive passes. A complete state contains the current board, side to move, previous-board residue, and consecutive-pass count. States with the same current board, player, and pass count are grouped; the analysis then asks whether changing only the reachable residue changes the set of legal placements.

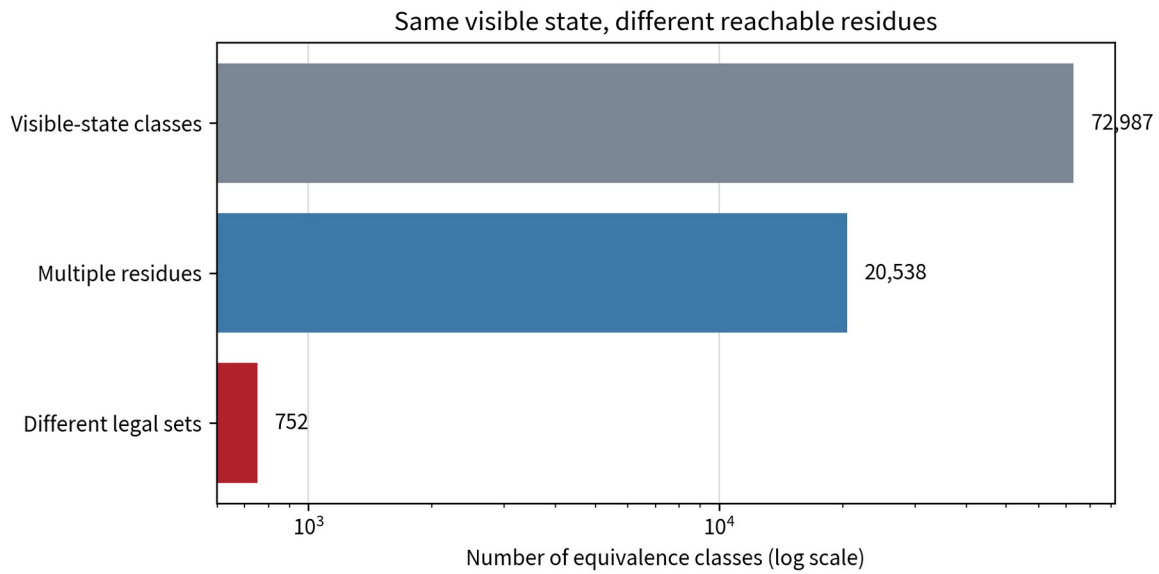


Figure 4. Visible-state classes in which reachable residues and legal sets differ.

Class	Count	Meaning
Complete states	132,161	Reachable states including residue
Directed legal edges	420,710	Placements and passes
Visible-state classes	72,987	Same board, player, and pass count
Classes with multiple residues	20,538	Different previous boards
Classes with different legal sets	752	Residue alone changes legality
Ko-sensitive complete states	768	Exactly one placement is blocked
Maximum distinct legal sets	3	For one visible-state class

The legality of White (0, 1) is not determined by the current board alone

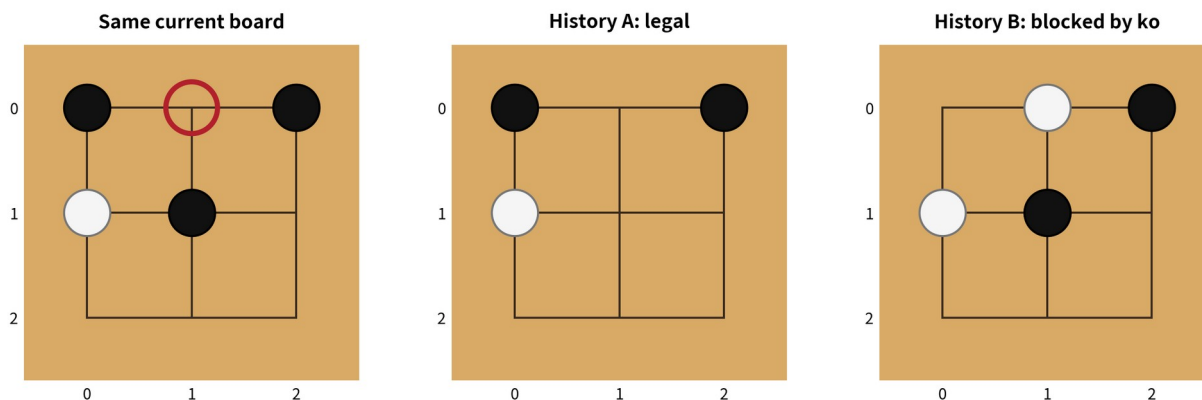


Figure 5. A shortest five-move witness pair: the same current board, different legality at White (0, 1).

This is not a result about predictive accuracy. It is a result about the information that must be present to decide whether an action is legal now. The two shortest witness histories reach the same board with the same player to move after five moves, yet White (0, 1) is legal in one state and forbidden in the other because it would recreate the latter state's previous board.

3.3 Design rules extracted from exhaustive search

Ablation	Observed change	Design judgment
Remove Renderer ordering	15,686 → 245,560 nodes	Separate correctness from cost
Remove certified reduction	32 → 245,560 nodes	Proven phenotypes may reduce candidates
Remove previous-board residue	One forbidden action enters 768 complete states	Residue is required for exact legality
Use one board instead of full history	Simple-ko judgment is retained	Store only what the selected law needs

4. WRRRA Go 0.1: a playable algorithm

4.1 From exhaustive proof to bounded action

Full enumeration validates the state definition and rule implementation on 3×3 Go, but it cannot close the entire state space of 9×9 or larger boards. WRRRA Go 0.1 retains the verified transition system, scores explicit structural properties of every legal candidate, and lets MCTS repeatedly compare candidates under a bounded computation budget. The same program runs controlled 5×5 matches and interactive 9×9 human-versus-engine games.

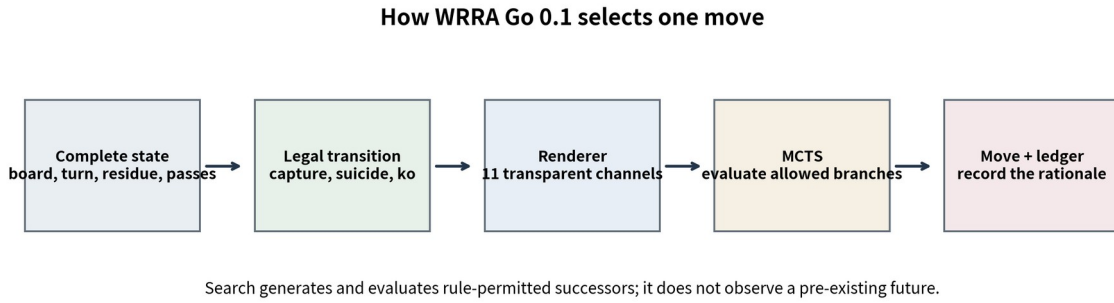


Figure 6. The end-to-end decision path in WRRRA Go 0.1.

4.2 Complete state and legal transition

$$S_t = (B_t, p_t, \rho_t, c_t, n_t), \quad \rho_t = B_{t-1}$$

Symbol	Component	Why it is needed
B	Current Black/White occupancy	Groups, liberties, capture, and scoring
p	Side to move	Distinguishes actor and opponent
ρ	Previous-board array	Rejects immediate simple-ko repetition
c	Consecutive-pass count	Terminates after two passes
n	Move number	Ledger and operational search boundary

$$T(S_t, a) = S_{t+1} \quad \text{iff} \quad a \in A_{\text{legal}}(S_t; L_D, \rho_t)$$

The placement function first checks vacancy, then removes adjacent opponent groups whose liberties fall to zero. It rejects the action if the newly placed player's group has zero liberties and no capture rescued it. It rejects simple ko when the resulting board equals p. A pass leaves the board unchanged while updating residue and the consecutive-pass counter. Two consecutive passes trigger area scoring.

4.3 Eleven transparent Renderer channels

The Renderer uses eleven explicit channels rather than hidden neural-network parameters. The weights are fixed, not learned. The score never decides legality: the rule engine first closes the legal candidate set, after which Renderer scores influence search priors and rollout order. Every candidate's score and phenotype are written to the ledger.

$$R(a \mid S) = \sum_{i=1}^{11} w_i f_i(S, a)$$

Channel	Computation	Weight
capture	Number of opponent stones captured immediately	+3.20
rescue	Structural rescue of adjacent friendly stones in atari	+2.40
atari	Opponent stones left with one liberty	+1.35
connection	Connection of two or more friendly groups	+0.72
cut pressure	Simultaneous contact with two or more opponent groups	+0.42
liberties	Liberties of the new group, capped at six	+0.20
self atari	One-liberty placement without capture	-3.25
eye fill	Filling a friendly eye candidate without capture	-2.10

Channel	Computation	Weight
area delta	Immediate change in small enclosed regions	+0.32
position	Board-size-dependent opening preference	+0.28
pass readiness	End when the opponent passed and the score is favorable	+1.00

4.4 MCTS integration

MCTS is an established search algorithm. WRRRA's contribution is not to rename it, but to provide a sufficient state, exact legal transition, explicit Renderer prior, guided rollout, phenotype labels, and an auditable ledger within one execution contract. Uniform MCTS with the same simulation count is used as a control so that the difference introduced by the Renderer can be measured directly.

$$a^* = \arg \max_a \left(Q(S, a) + c P_R(a | S) \frac{\sqrt{N(S)}}{1 + N(S, a)} \right)$$

1. Generate all legal placements and pass transitions from the current complete state.
2. Normalize Renderer scores with a softmax to produce candidate priors.
3. Descend the tree using mean value Q plus a prior-weighted exploration term.
4. At a new node, run a bounded rollout biased toward high-ranked Renderer candidates.
5. Backpropagate terminal or bounded area score from the root player's perspective.
6. Select the most-visited action and record visits, value, prior, Renderer score, and phenotype.

4.5 Running the program

Purpose	Command	Output
9×9 human vs engine	<code>python src/wrra_go_0_1.py play --size 9 --human B</code>	Interactive coordinate input
Deterministic rule tests	<code>python src/wrra_go_0_1.py test</code>	Ko, capture, suicide, termination

Purpose	Command	Output
Paired benchmark	<code>python src/wrra_go_0_1.py benchmark</code>	Machine-readable JSON ledger
9×9 self-play	<code>python src/wrra_go_0_1.py selfplay --size 9</code>	JSON decision ledger and SGF

Human moves use standard Go coordinates such as D4 and the word pass; column I is skipped. The default 72 simulations are an executable baseline. Larger budgets examine more candidates per move. The rule engine accepts 13×13 and 19×19 boards, while the quantitative tests in this release focus on 5×5 and 9×9.

5. Execution validation

5.1 Deterministic rule tests

Test	Result
Reproduce the same-visible-board witness pair	PASS
Simple ko blocks restoration of the previous board	PASS
Removing residue admits the repetition	PASS
Different residues change legality on the same board	PASS
Two consecutive passes terminate the game	PASS
Capture removes a surrounded group	PASS
Suicide is forbidden	PASS
Area scoring is deterministic	PASS
Renderer does not mislabel an open region as territory	PASS
Coordinate conversion round trip	PASS

The key regression test replays the shortest five-move witness found by Game 0.2 inside the engine's independent immutable state object. Both histories reach the same board with the same player to move, but their previous boards differ. White (0, 1) is legal in one state and rejected by simple ko in the other. This confirms that the state condition discovered by exhaustive analysis survives in the playable code.

5.2 Paired-colour 5×5 benchmark

WRRM MCTS and each control alternated Black and White. Each move used 48 simulations. Uniform MCTS retained the same tree-search mechanism and computation budget but removed Renderer priors and guided rollouts. The random control selected one legal action uniformly.

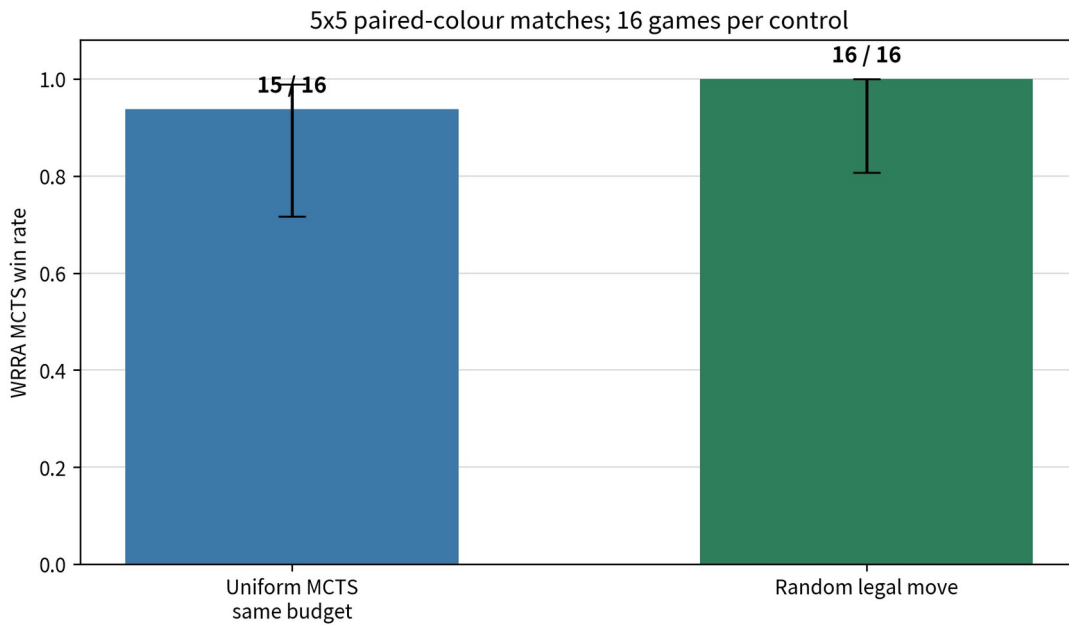


Figure 7. WRRM MCTS against controls under identical rules and alternating colours; error bars are Wilson 95% intervals.

Control	Games	WRRM W-L	Win rate	Wilson 95% interval	Mean moves
Uniform MCTS	16	15-1	93.8%	71.7-98.9%	34.8
Random	16	16-0	100.0%	80.6-100.0%	37.9

The justified conclusion is that the fixed Renderer produced an effective action-selection bias in this controlled sample and won more games than equal-budget uniform search. This sample does not establish superiority to professional engines or generalize playing strength to 19×19 . Those are separate experiments that can be added without changing the execution contract.

5.3 A complete 9×9 self-play record

Two WRRRA MCTS agents, each using 24 simulations per move, played a 9×9 game that ended after 82 moves by two consecutive passes. Black area was 38, White area was 41, and komi was 5.5. The Black-margin score was -8.5; the winner was White.

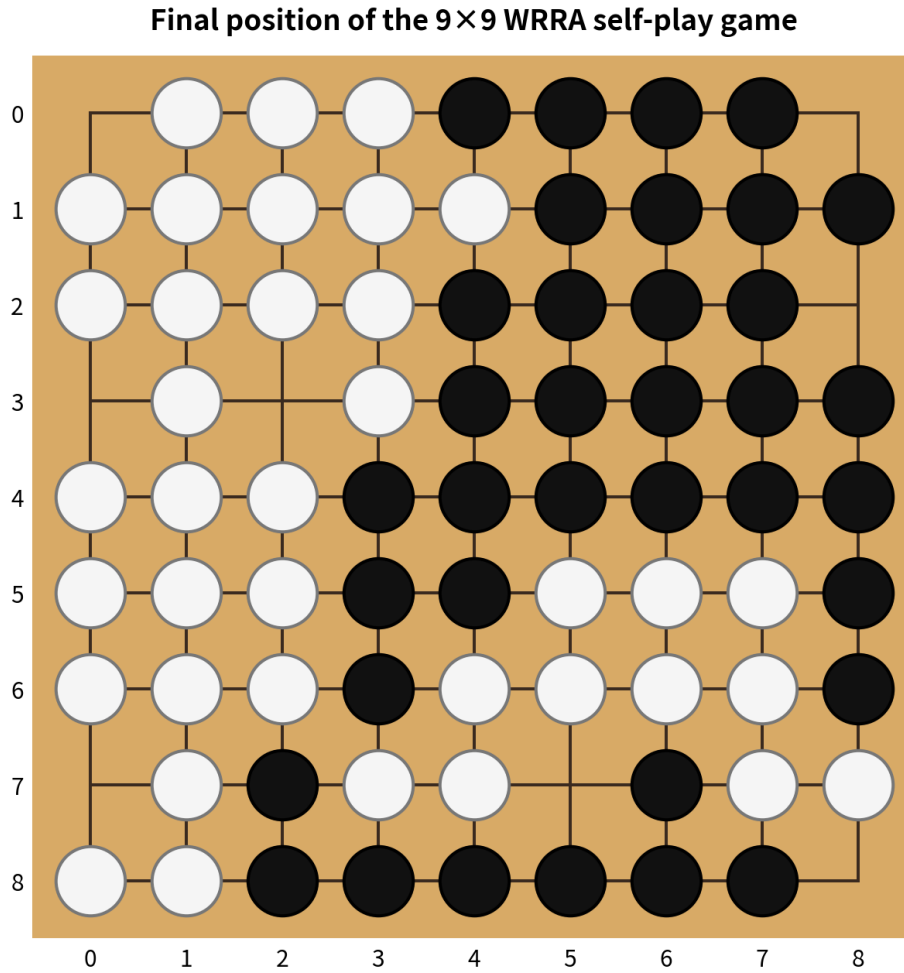


Figure 8. Final position of the 9×9 WRRRA self-play game; the complete SGF is included.

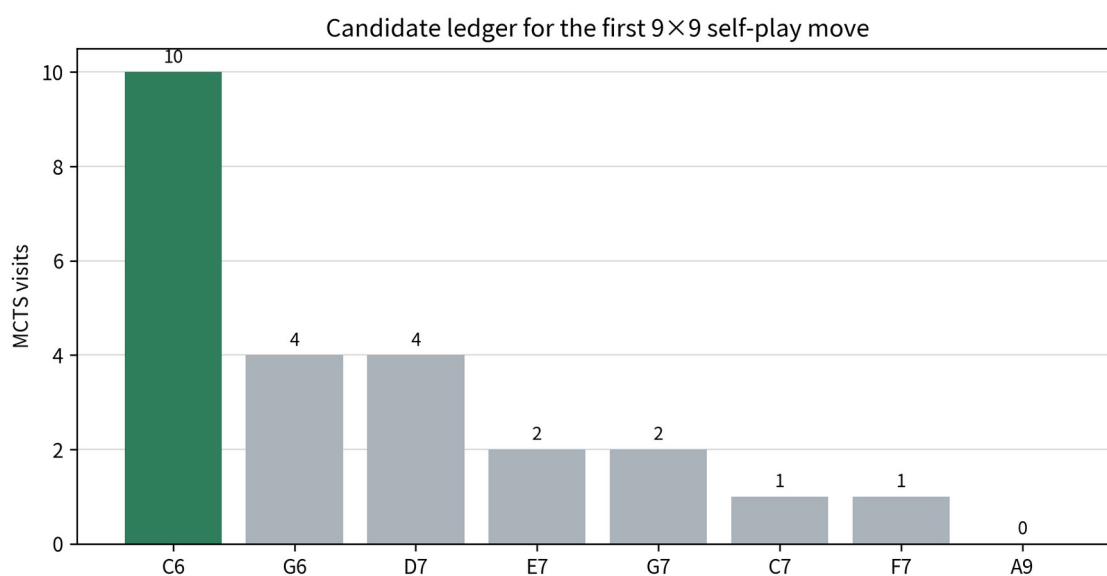


Figure 9. Candidate visits recorded for the first self-play move.

Move	Visits	Mean value	Prior	Renderer score	Phenotype
C6	10	+0.000	0.0142	+1.080	extension
G6	4	+0.500	0.0142	+1.080	extension
D7	4	+0.000	0.0142	+1.080	extension
E7	2	+0.000	0.0142	+1.080	extension
G7	2	+0.000	0.0142	+1.080	extension
C7	1	-1.000	0.0142	+1.080	extension
F7	1	-1.000	0.0142	+1.080	extension
A9	0	+0.000	0.0092	+0.493	extension

5.4 Complete move list

Move no.	Black	White
1	C6	G6

Move no.	Black	White
2	F7	D7
3	D3	C7
4	F8	F3
5	E7	D8
6	G7	B7
7	G8	B6
8	H7	F4
9	H8	G4
10	E6	D6
11	F6	G3
12	E5	E3
13	E4	C5
14	G5	B5
15	G9	B8
16	H6	E2
17	D4	H4
18	F5	B4
19	E9	E8
20	D5	H3
21	C2	D2
22	C1	C8

Move no.	Black	White
23	J6	J9
24	H9	H2
25	J5	C9
26	H5	D9
27	D1	B3
28	F1	A7
29	E1	C3
30	G1	A4
31	J8	B2
32	J4	B1
33	A2	F9
34	J3	A1
35	H1	A3
36	E9	A5
37	F9	J2
38	G2	C4
39	A9	B9
40	G6	A8
41	pass	pass

The SGF stores the same sequence in standard coordinates. The JSON ledger stores each move's leading candidates, visits, mean value, prior, Renderer score, phenotype, and computation time. The selected action and its recorded rationale can therefore be re-examined independently.

6. Scientific and engineering value

6.1 A connected chain of evidence

1. The same Core contract is applied to different games while preserving their different residue requirements. Integration means a shared test method, not erasure of domain differences.
2. The work moves from constructed examples to complete reachability graphs, exact counterexamples, and value-preservation checks. Its explanations are therefore corrigible by computation.
3. State correctness is separated from search efficiency. Removing residue changes legality; removing Renderer guidance can preserve value while increasing cost.
4. When exhaustive search becomes impossible, the verified transition system is retained and coupled to bounded approximate search. Exact small-system proof is connected to larger-board action.
5. Renderer channels, weights, candidate visits, and phenotypes are public. A move can be traced to explicit structural evidence rather than only a final output.
6. The state–law–residue–Renderer–ledger interface is reusable for Omok, Go, Janggi, chess, and other deterministic games without modifying the frozen Core.

6.2 Additional strengths revealed by the experiments

Strength	Evidence in this release	Meaning beyond games
Automatic state-aliasing detection	752 residue-dependent visible-state classes	Find different permitted actions behind the same observation
Proof-carrying reduction	Zero Connect-K value mismatches	Use local phenotypes as certified computation gates

Strength	Evidence in this release	Meaning beyond games
Correctness/cost separation	Residue and Renderer ablations	Design memory and computation budget independently
Scale transition	3×3 exact validation to 9×9 MCTS	Connect exhaustive tests to bounded operation
Explainable action	Eleven channels and per-candidate ledger	Reinspect why an action was selected
Core invariance	No Core modification across 0.1, 0.2, and Go 0.1	Extend domains while preserving the upper contract

The order of validation matters. First establish rule correctness, then state sufficiency, then lossless reduction, and only then evaluate bounded-search performance. The same pattern can connect small exact models with operational systems in power-grid analysis, biological regulation, and agent behavior, where total-state enumeration is unavailable.

6.3 What is established and what remains open

Established in this release	Next extension
Legal actions and transitions under simple ko	Compressed residue for positional superko
5×5 controlled matches and 9×9 execution	Multi-seed 9×9 tournaments and timed 19×19 play
Fixed transparent Renderer channels	Weight ablation and calibrated optimization
Executable integration with MCTS	Rated GTP matches against external engines
Area score, SGF, and JSON ledgers	Dead-stone handling and separate Chinese/Japanese profiles

This table is not a retreat from the achieved result. It fixes the input and output of the next experiments. Version 1.0 is already a standalone program that a person can play; stronger tournament performance is a measurable extension on the same architecture.

7. Reproducibility and release

7.1 File lineage and SHA-256 manifest

File	Role	SHA-256
wrra_game_0_1.py	Constructive Omok and Go analysis	e5bdf92a69b7bd50c7444d099cbb5c0848fd7bebd2bead2d78d5337825d36bd6c
wrra_game_0_1_results.json	Game 0.1 result ledger	4de1c203b65a0b85dd71cc2e2988d28befedbbdc2900e25a713383e86d3c2dc4
wrra_game_0_2.py	Connect-K and 3×3 Go exhaustive analysis	ec010ec0de5f045dcf47956a9cfd223ded06a12a5933a6c51ffacfb4af95e2b
wrra_game_0_2_results.json	Game 0.2 result ledger	bed55e8efda6f8972be0cecc04f929d900fd4292278fdce82262500bc0abbab8
wrra_go_0_1.py	Playable engine and controlled evaluation	8549c63f35fbf13d6c4d0ae48ed797132bfca8edf69177d84d35cf3d4f6328c9
wrra_go_0_1_benchmark.json	Paired-colour benchmark	1368144e9df8a02ef391b4ddb8ed9c1a39eb45a46dba579afc5a4a162d20b98a
wrra_go_0_1_rule_tests.json	Deterministic rule regression tests	c1e6c40a6f7803285b9ab0d45f493cc136ff648b62cb9d6b7809a433ea33e376
wrra_go_0_1_selfplay_9x9.json	Complete 9×9 decision ledger	fa340a48ed25887b3bc54cc86f48df9531bfc12e3f76c5a11ace9ab101093195
wrra_go_0_1_selfplay_9x9.sgf	9×9 self-play SGF	6d1c86ac405e54f514c9742d93a194fc0049fc10b63b6a30219c1564df297680

7.2 Reproduction sequence

1. Run `python src/wrra_go_0_1.py test` and confirm that all ten rule tests are true.
2. Run `python src/wrra_game_0_2.py` and confirm 6,036,001 Connect-K states and zero value mismatches.
3. In the same ledger, confirm 132,161 complete 3×3 Go states and 752 residue-dependent visible-state classes.
4. Run `python src/wrra_go_0_1.py benchmark --size 5 --paired-games 8 --simulations 48`.
5. Run `python src/wrra_go_0_1.py selfplay --size 9 --simulations 24` to create JSON and SGF outputs.
6. Start a human game with `python src/wrra_go_0_1.py play --size 9 --human B`.

The recorded benchmark runtime is 137.78 seconds and will vary by environment. State counts, legality tests, fixed seeds, and recorded outcomes are reproducible under the same Python behavior and inputs. The archive includes the exhaustive 0.2 ledger so that its results can be inspected without repeating the full enumeration.

7.3 Recommended repository and Zenodo metadata

Field	Value
Title	WRRR Game 1.0: Minimal Sufficient State, Exhaustive Validation, and a Playable Go Engine
Subtitle	From Constructive Omok and Go Analysis to WRRR Go 0.1
Creator	Wonsik Choi
Contact	janefather@gmail.com
Resource type	Software and technical report
Language	English; Korean companion report included
Version	1.0
Keywords	WRRR, Go, Baduk, Omok, Connect K, MCTS, minimal sufficient state, residue, Renderer, exhaustive search, explainable game AI, complex systems
Related identifier	WRRR Core 1.0 — https://doi.org/10.5281/zenodo.22650956
License	Reports, documentation, and data: CC BY 4.0; source code: MIT

Release description

This release integrates the WRRR Game research program from constructive game analysis to exhaustive finite-state validation and a playable Go engine. Game 0.1 identifies an Omok forced-win

phenotype and a simple-ko residue counterexample. Game 0.2 exhaustively enumerates 6,036,001 reachable Connect-K states and 132,161 reachable 3×3 Go states, finding zero minimax-value mismatches for a certified Renderer reduction and 752 visible-state classes whose legal actions depend on previous-board residue. WRRR Go 0.1 reuses the verified transition system in a playable 9×9 engine, combining a transparent eleven-channel Renderer with Monte Carlo Tree Search. The archive includes source code, machine-readable ledgers, deterministic rule tests, paired-control matches, and an SGF self-play record. The work evaluates rule-permitted continuations; it does not describe residue or search as prediction of a pre-existing future.

7.4 Research lineage

WRRR Core 1.0: Frozen Domain-Agnostic Execution Architecture. DOI: 10.5281/zenodo.22650956.

WRRR Game 0.1: Constructive complexity in Omok and Go. 16 September 2026.

WRRR Game 0.2: Exhaustive Connect-K validation and residue sufficiency in Go. 17 September 2026.

WRRR Go 0.1: The playable engine included in this report. 17 September 2026.

Conclusion

This work does not stop at the statement that simple rules can create complex games. In Connect K it tests, over every reachable state, which Renderer phenotypes can act as certified computation reductions without changing value. In Go it identifies, through exhaustive counterexamples, which residue must enter the state for exact legality. It then places the same transition machinery in a playable 9×9 agent that selects moves and records both SGF and candidate-level ledgers.

The central achievement of WRRR Game 1.0 is therefore a single reproducible execution system: preserve rule correctness with a minimal sufficient state, expose computationally relevant structure with a transparent Renderer, and select actual actions through bounded search.